

CMP4272 Data Structures and Algorithms



BIRMINGHAM CITY
University

Lecture 7 - Algorithms

Data Structures

- A way to store and organize data to facilitate access and modifications.
- Using the appropriate data structure or structures is an important part of algorithm design.
- No single data structure works well for all purposes, and so you should know the strengths and limitations of several of them.

Algorithm Definition

- A step-by-step procedure designed to solve a specific problem or perform a particular task.



Algorithm: Tea

Input:

Tea bag, kettle, water, milk, sugar, spoon, cup

Output:

Tea

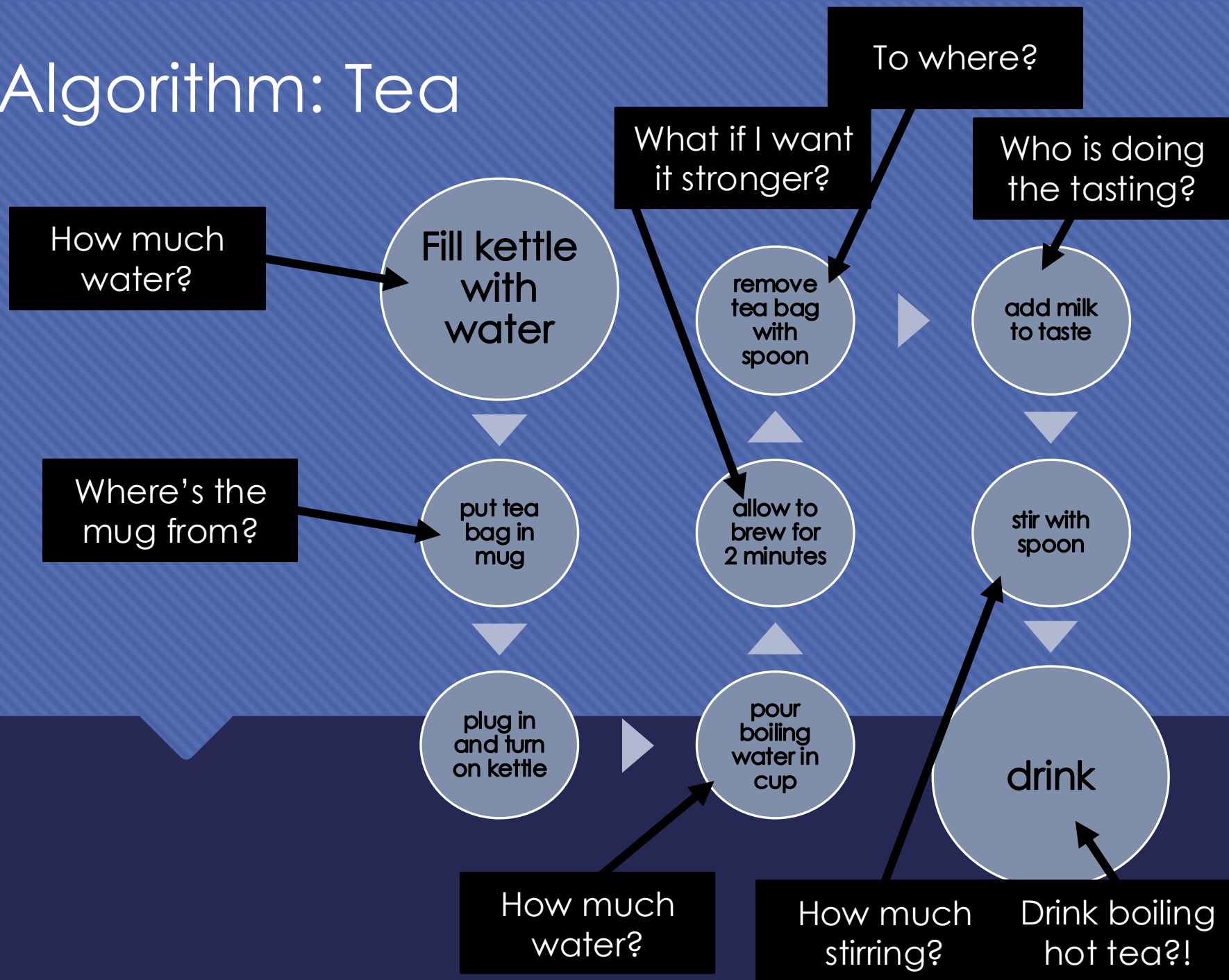
Algorithm:

Steps for making tea



BIRMINGHAM CITY
University

Algorithm: Tea



Algorithm EvenOdd

Input: An integer number 'n'

Output: "Even" if 'n' is even, "Odd" if 'n' is odd

Algorithm:

1. Read the integer number 'n'.
2. Check if 'n' modulo 2 is equal to 0:
 - If true, 'n' is even. Output "Even".
 - If false, 'n' is odd. Output "Odd".
3. End



Algorithm Characteristics

- 1.**Finite**: An algorithm must terminate after a finite number of steps.
- 2.**Definite**: Each step in the algorithm must be precisely and unambiguously defined.
- 3.**Effective**: Algorithms must be executable and practical, meaning they can be carried out using available resources within a reasonable amount of time.
- 4.**Input/Output**: An algorithm takes zero or more inputs and produces one or more outputs.
- 5.**Correctness**: An algorithm should produce the correct output for all valid inputs.
- 6.**Deterministic**: Given the same input, an algorithm should always produce the same output.

Algorithm Problem Examples

The following are examples of problems that require algorithms to solve:

- Reverse the contents of a list in-place.
- Count how many times a value occurs in a list.
- Find a shortest path between two nodes in a weighted graph.

Algorithm, Pseudocode, Program

Algorithms
instance
diagramm

Algorithm of linear search :

1. Start from the leftmost element of arr[] and one by one compare x with each element of arr[].
2. If x matches with an element, return the index.
3. If x doesn't match with any of elements, return -1.

ways, for
or other

Pseudocode is used
It doesn't have specific
should be interpreted

Pseudocode for Linear Search :

```
FUNCTION linearSearch(list, searchTerm):  
  FOR index FROM 0 -> length(list):  
    IF list[index] == searchTerm THEN  
      RETURN index  
    ENDIF  
  ENDOLOOP  
  RETURN -1  
END FUNCTION
```

algorithm for a program.
language agnostic, but
experts.

A program is a set

computer to follow.

```
int search(int arr[], int n, int x)  
{  
  int i;  
  for (i = 0; i < n; i++)  
    if (arr[i] == x)  
      return i;  
  return -1;  
}
```



BIRMINGHAM CITY
University

The Problem-Solving Process

There are 5 steps to writing your algorithm:

1. Specify the problem.
2. Split the problem into subproblems if possible.
3. Solve the subproblems: If you already have plans for solving them, use them. Otherwise, temporarily forget about the original problem and solve one subproblem at a time.
4. Combine the subproblem solutions into a solution for the original problem.
5. Test the solution to make sure it works.



Specifying the Problem

- Find the key words in a problem statement
- Identify inputs and outputs
- Give examples to clarify the problem statement.
- Remember the flow of an algorithm:

Input → Algorithm → Output



Inputs and Outputs

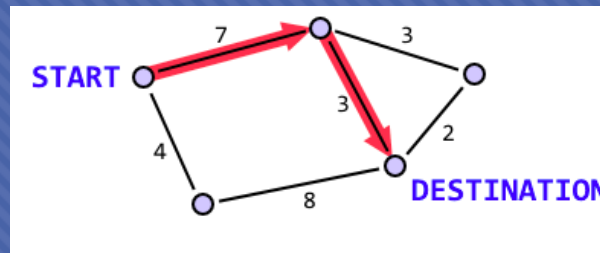
- Identify the inputs and outputs? Think of an example for each.

1. Reverse the contents of a list in-place.
2. Count how many times a value occurs in a list.
3. Find a shortest path between two nodes in a weighted graph.

In order to translate a problem into an algorithm to generate a solution you must be able to identify the inputs and outputs before getting into the technicalities of what the algorithm will do.

Inputs and Outputs Examples

1. Reverse the contents of a list in-place.
The list [1,2,3,4] will become [4,3,2,1].
2. Count how many times a value occurs in a list.
The value 7 occurs 3 times in [1,7,2,7,7,5,5,9].
3. Find a shortest path between two nodes in a weighted graph.



Inputs and Outputs Identified

1. Reverse the contents of a list in-place.

Input: The list to be reversed.

Output: None (we're handed a pointer so nothing is output here)

2. Count how many times a value occurs in a list.

Input: A list

Input: The value to be counted

Output: The number of occurrences as an integer

3. Find a shortest path between two nodes in a weighted graph.

Input: a weighted graph.

Input: the start node.

Input: the destination node.

Output: a path, represented as a list of nodes beginning with the start node, and ending with the destination node.



Inputs and Outputs Identified

It is imperative to specify inputs and outputs before writing an algorithm as these define the signature of the code whose variables are used throughout a program.

```
def reverse_list(lst):  
    # ...  
  
def count_occurrences(lst, value):  
    # ...  
  
def shortest_path(graph, node_from, node_to):  
    # ...  
  
def polygons_intersect(polygon1, polygon2):  
    # ...
```



Problem -> Subproblems

- Breaking down larger problems into subproblems
- This allows for the steps required in an algorithm to be clearly identified.
- This also helps find efficiencies in code.



Problem -> Subproblems

- Let's look at the counting example again.
- If the list we want to count from is [1,7,2,7,7,5,5,9] and we want to count the number of 7's

val

lst

- The problem of counting can be broken into three subproblems:
 1. Look at each value in *lst* one by one
 2. Detect occurrences of *val*.
 3. Count the detected occurrences.



The Problem-Solving Process Again

There are 5 steps to writing your algorithm:

1. Specify the problem.
2. Split the problem into subproblems if possible.
3. Solve the subproblems: If you already have plans for solving them, use them. Otherwise, temporarily forget about the original problem and solve one subproblem at a time.
4. Combine the subproblem solutions into a solution for the original problem.
5. Test the solution to make sure it works.



Plans

Plans are the basic building blocks of algorithms where each solves a specific part of a problem or subproblem rather than providing an entire solution. There are 8 common plans:

1. Do something for each value
2. Filter
3. Map
4. Collect results in a new list
5. Keep a running total
6. Keep a counter
7. Find something with a “linear search”
8. Keep track of a “best so far”

Plans - 1

Do something for each value: This requires a loop such as a for or while loop to iterate over all values.

```
for value in lst:  
    # ...
```

```
for i, value in enumerate(lst):  
    # ...
```

Plans - 2

Filter: Selecting values that meet specific conditions. An if statement is the most common way of filtering. Typical examples would be:

- Find the sum of the positive numbers in a list.
- Find the largest even number in a list.
- Count how many strings in a list are shorter than 4 characters.

```
for value in numbers:  
    if value > 0:  
        # ...
```

Plans - 3

Map: modify input values to give a desired output. This output is usually computed and stored in a new variable. Examples include:

- Find the sum of squares of a list
- Convert temperatures from Celcius to Fahrenheit

```
for value in numbers:  
    square = value ** 2  
    # ...
```

Plans - 4

Collect results in a new list: when filtering or mapping must be stored without destroying the original data. An example would be converting temperatures in Celsius to Fahrenheit and returning these values in a new list call *Fahrenheit*.

- 1) Before the loop, create a new empty list for the result.
- 2) Within the loop, append each individual result to the new list.
- 3) At the loop end return the list of results or use it another way.

```
def celsius_to_fahrenheit(temperatures_c):  
    temperatures_f = []  
    for celsius in temperatures_c:  
        # formula for converting Celsius to Fahrenheit  
        fahrenheit = 1.8 * celsius + 32  
        temperatures_f.append(fahrenheit)  
    return temperatures_f
```

Plans - 5

Keep a running total: calculate the sum of a list of numbers which may have some conditions. An example would be to find the sum of the positive numbers in a list.

- 1) Before the loop, create a variable for the running total, and initialise it to 0.
- 2) Each time we “see” a value, update the running total by adding it on.
- 3) After the loop, the total is complete, so return it (or use it some other way).

```
def sum_of_positives(numbers):  
    total = 0  
    for value in numbers:  
        if value > 0:  
            total += value  
    return total
```

Plans - 6

Keep a counter: count how many of something there are. An example could be to count how many strings in a list have less than 4 characters.

- 1) Before the loop create a “counter” variable and initialise to 0.
- 2) Each time we “see” something being counted, increment the counter.
- 3) After the loop, the counter is complete.

```
def count_short_strings(strings):  
    counter = 0  
    for value in strings:  
        if len(value) < 4:  
            counter += 1  
    return counter
```

Plans - 7

Linear Search: based on a description find something and return all or only the first instance. An example could be to find the negative numbers in a list.

- 1) If we found what we're looking for, then we already know the correct result, so we can return it immediately so the function "goes home early".
- 2) Therefore, if we get to the end of the loop, we must not have found it, so handle this case separately so the function "goes home late".

```
def find_first_negative(numbers):  
    for value in numbers:  
        if value < 0:  
            return value  
  
    raise ValueError('No negative number found')
```



Plans - 8

Best so far: used to find the 'best' of something. Examples would be the highest number, the longest string or the Person class with the highest score.

- 1) Before the loop, declare a "best so far" variable — initially this is the first thing we "see", at index 0.
- 2) Each time we "see" something "better", we update the "best so far".
- 3) After the loop, the "best so far" is the true "best".

```
def find_largest_number(numbers):  
    largest = numbers[0]  
    for value in numbers:  
        if value > largest:  
            largest = value  
    return largest
```

Exercise: Recognising Plans

Which plans are being used here?

1. Name the plans used in the following code, and specify the problem this function solves.

```
def exercise1(numbers):  
    for i, value in enumerate(numbers):  
        numbers[i] = 10 * value
```

2. Name the plans used in the following code, and specify the problem this function solves.

```
def exercise2(numbers):  
    total = 0  
    for value in numbers:  
        cube = value ** 3  
        total += cube  
    return total
```

Exercise: Recognising Plans

Which plans are being used here?

3. Name the plans used in the following code, and specify the problem this function solves.

```
def exercise3(numbers, cutoff):  
    for i, value in enumerate(numbers):  
        if value >= cutoff:  
            return i  
    return -1
```

4. Name the plans used in the following code, and specify the problem this function solves.

```
def exercise4(strings, max_length):  
    counter = 0  
    for value in strings:  
        if len(value) > max_length:  
            counter += 1  
    return counter
```

The Problem Solving Process Again

There are 5 steps to writing your algorithm:

1. Specify the problem.
2. Split the problem into subproblems if possible.
3. Solve the subproblems: If you already have plans for solving them, use them. Otherwise, temporarily forget about the original problem and solve one subproblem at a time.
4. Combine the subproblem solutions into a solution for the original problem.
5. Test the solution to make sure it works.



Combine Subproblems

Starting with the function signature work down the list of subproblems and solve each to combine into a problem solution.

```
def count_occurrences(lst, value):  
    # ...
```

The first plan is to "do something *for each value*":

```
def count_occurrences(lst, value):  
    for x in lst:  
        # ...
```

The second plan is to "*filter* for what we want":

```
def count_occurrences(lst, value):  
    for x in lst:  
        if x == value:  
            # ...
```

The third plan is to "use a *counter* variable":

```
def count_occurrences(lst, value):  
    counter = 0  
    for x in lst:  
        if x == value:  
            counter += 1  
    return counter
```

The Problem Solving Process Again

There are 5 steps to writing your algorithm:

1. Specify the problem.
2. Split the problem into subproblems if possible.
3. Solve the subproblems: If you already have plans for solving them, use them. Otherwise, temporarily forget about the original problem and solve one subproblem at a time.
4. Combine the subproblem solutions into a solution for the original problem.
5. Test the solution to make sure it works.

Test the Solution

Does it actually do what it's meant to? Originally when developing at scale Microsoft had 2 testers working with each developer to ensure quality. Often this is stripped back and in many small or medium enterprises the developer is the tester. Make sure you use realistic and ridiculous cases as users will often put code to both good and bad use!

```
>>> count_occurrences([1, 7, 2, 7, 7, 5, 9], 7)
3
>>> count_occurrences(['hello', 'world'], 'hello')
1
>>> count_occurrences([1, 3, 2, 5, 4], 10)
0
```

THANK YOU